

Roles, trust policies, cross-account and the confused deputy

Who may become this role, what it may then do, and the attack that exists in the gap between them.

A role is **two policies**. One says who may become it. The other says what it may then do.

The trust policy

- ◆ A resource-based policy on the role. The one type where **Principal is mandatory**.
- ◆ Action is **sts:AssumeRole**, AssumeRoleWithSAML or AssumeRoleWithWebIdentity.
- ◆ **Principal**: an account, a user or role ARN, an AWS **Service**, or a **Federated** provider.
- ◆ Conditions belong here: **sts:ExternalId**, **aws:PrincipalOrgID**, **aws:SourceArn**, MFA.
- ◆ **Principal "*" with no condition** means any AWS account may try. Access Analyzer flags it.

ON THE EXAM

Cross-account needs both halves: this policy naming the caller, and the caller's own sts:AssumeRole.

The :root nuance that catches everyone

- ◆ A Principal of `arn:aws:iam::111122223333:root` does not mean the root user.
- ◆ It **delegates to that whole account** – any principal in it permitted to call `sts:AssumeRole` may assume the role.
- ◆ You have handed the decision to the other account's administrators. Often right between your own accounts.
- ◆ Rarely right for a third party: name the specific role ARN **and** require an external ID.
- ◆ But naming a role ARN breaks if the vendor recreates it – which is why vendors prefer the account form.

ON THE EXAM

Account-level trust plus a permissive policy on the other side widens a narrow-looking role.

Three ways to assume

API	CALLER	TRUST POLICY PRINCIPAL
<code>sts:AssumeRole</code>	An AWS principal – user, role, service	AWS account, user, role, or Service
<code>sts:AssumeRoleWithSAML</code>	A user federated through a SAML 2.0 IdP	Federated – the SAML provider ARN
<code>sts:AssumeRoleWithWebIdentity</code>	A user or app via an OIDC provider	Federated – the OIDC provider ARN

ON THE EXAM

Cognito identity pools and EKS IRSA both use AssumeRoleWithWebIdentity underneath. So does GitHub Actions OIDC.

Session duration and role chaining

- ◆ A role has a **maximum session duration** from 1 to 12 hours. The caller asks for DurationSeconds up to that ceiling.
- ◆ **Role chaining** – assuming a role using credentials from an already-assumed role – is **capped at 1 hour** and cannot be extended.
- ◆ Ask for more than an hour while chaining and the call **fails**, rather than being silently truncated.
- ◆ The restriction applies to the **CLI and API**. The console's switch-role feature is not chaining, because it uses your original credentials.
- ◆ The fix for a long-running job is to **not chain** – assume the target role directly from a stable identity.

ON THE EXAM

A job that runs fine for 55 minutes and then dies is role chaining, almost every time.

The confused deputy, and defence one

- ◆ **The problem:** a third party serving many customers is tricked by one into acting in another's account.
- ◆ **The defence:** `sts:ExternalId` in your trust policy, matching a value the vendor generates for you.
- ◆ The vendor supplies your external ID only when acting for you. Another customer cannot make them.
- ◆ It is **not a secret**. It defeats confusion, not theft.
- ◆ The same key secures **EKS Pod Identity associations**.

ON THE EXAM

The vendor generates the value. If they let you choose it, they have missed the point.

Defence two: when the deputy is an AWS service

- ◆ The same problem with **AWS service principals**: a service may be pointed at your resource.
- ◆ **aws:SourceArn** pins the resource it is acting for. **aws:SourceAccount** pins the account.
- ◆ Classic shape: a KMS key or SNS topic policy allowing a service principal, scoped by SourceArn.
- ◆ Module 1: these keys are **not supported on CloudTrail Lake event data store key policies**.
- ◆ IAM Roles Anywhere supports them for its service principal too.

ON THE EXAM

External ID: a third party you contracted. SourceArn: a service AWS runs.

External ID, or SourceArn?

STS:EXTERNALID

The deputy is a company

- A SaaS vendor or partner account
- Goes in the role's trust policy
- The vendor generates the value
- Multi-tenant vendor, many customers

AWS:SOURCARN / SOURCEACCOUNT

The deputy is an AWS service

- cloudtrail, sns, s3, lambda and friends
- Goes in the resource policy or key policy
- You supply your own resource ARN
- One service, many customers' resources

THE TIEBREAKER

Both answer 'acting for whom'. The difference is whether the deputy is a company you contracted or a service AWS runs.

Session tags and source identity

- ◆ **sts:TagSession** attaches tags to the session, usable as **aws:PrincipalTag**. This is how ABAC federates.
- ◆ **Transitive tags** survive role chaining, so an IdP attribute follows the identity.
- ◆ **sts:SourceIdentity** carries the original human through the chain and **cannot be changed**.
- ◆ That turns an unattributable assumed-role session in CloudTrail into a named person.
- ◆ The trust policy must permit **TagSession** and **SetSourceIdentity** separately.

ON THE EXAM

Source identity answers 'which human' in an estate of shared roles.

iam:PassRole, the quiet privilege escalation

- ◆ **PassRole hands a role to a service** – an instance profile, a Lambda execution role.
- ◆ **Unscoped PassRole plus ec2:RunInstances is administrator access.**
- ◆ Scope it: **Resource** listing the specific role ARNs. Never a wildcard.
- ◆ Add **iam:PassedToService** so a Lambda role cannot be attached to an instance.
- ◆ Access Analyzer surfaces an over-broad PassRole. Module 25.

ON THE EXAM

PassRole turns a limited role into any role. Put it on the escalation checklist.

IAM Roles Anywhere

- ◆ Temporary AWS credentials for **on-premises workloads** using an **X.509 certificate**.
- ◆ Three objects: a **trust anchor** (your CA), a **profile**, and the **role**.
- ◆ The role must trust **rolesanywhere.amazonaws.com**; credentials come from **CreateSession**.
- ◆ Certificates: **X.509v3**, key usage including **Digital Signature, SHA-256 or stronger**. MD5 and SHA-1 rejected.
- ◆ **Empty certificate subjects are not supported** – the subject is how callers are audited.

ON THE EXAM

Same idea as the IoT Core credential provider: a trusted certificate, exchanged for credentials.

A retailer engages a SaaS cost-optimisation vendor. It creates a role whose trust policy allows Principal `arn:aws:iam::444455556666:root`, the vendor's account, with action `sts:AssumeRole` and no conditions. The role has `ReadOnlyAccess`.

A security reviewer objects. The engineer replies that only the vendor's account is trusted, and the vendor is a reputable company with SOC 2 attestation.

What is the reviewer's strongest objection?

- ☐ A Another of the vendor's customers could cause the vendor to act in this account; require `sts:ExternalId`
- ☐ B The trust policy grants the vendor's root user access; name a specific IAM user instead
- ☐ C `ReadOnlyAccess` is too broad for a cost tool; scope it to Cost Explorer actions
- ☐ D Cross-account roles require MFA in the trust policy to be compliant

◆ PAUSE AND DECIDE

What is the reviewer's strongest objection?

- A

 Confused deputy; require sts:ExternalId
Correct. The risk is not the vendor's honesty – it is a configuration issue at a multi-tenant vendor letting one customer point them at another's account.
- B

 It grants the vendor's root user
The :root nuance, misread. That ARN delegates to the whole account, not to its root user – which is a real concern, and not the one this option states.
- C

 ReadOnlyAccess is too broad
A fair least-privilege point and the weaker objection. Narrowing the permissions does not stop the wrong customer triggering them.
- D

 MFA is required in cross-account trust policies
No such requirement, and MFA is meaningless for a machine-to-machine vendor integration. Nobody is at a keyboard.

THE SIGNAL

The engineer's defence – the vendor is reputable – is answering the wrong risk. Confusion is not dishonesty.

A company's SNS topic policy allows Principal Service cloudtrail.amazonaws.com to publish, with no conditions. The topic receives CloudTrail delivery notifications.

A researcher demonstrates that anybody who creates a CloudTrail trail in any AWS account can configure it to publish to this topic, flooding it with notifications the company must pay for and process.

What should the company add?

- ☐ A sts:ExternalId in the topic policy, generated for the company
- ☐ B aws:PrincipalOrgID, so only principals in the company's organisation may publish
- ☐ C A Deny for all principals except the CloudTrail service, evaluated first
- ☐ D aws:SourceArn matching the company's trail ARN, or aws:SourceAccount matching its account

◆ PAUSE AND DECIDE

What should the company add?

A sts:ExternalId in the topic policy

Wrong defence for this deputy. External ID is an sts condition for role assumption by a third-party company, not a resource policy control for a service principal.

C Deny all except the CloudTrail service

*Restates what the policy already does. CloudTrail *is* the principal making the call - the problem is whose trail it is acting for.*

B aws:PrincipalOrgID

The principal here is the CloudTrail service, which is not a member of anyone's organisation. The condition would not match and would break delivery entirely.

D aws:SourceArn or aws:SourceAccount

Correct. These pin the resource the service is acting on behalf of, so the service principal only carries this permission for the company's own trail.

THE SIGNAL

The principal is right and the resource behind it is wrong. That gap is exactly what SourceArn closes.

A security scanning tool runs on EC2 in a tooling account. Its instance role assumes a cross-account audit role in each of 90 member accounts, and each scan takes about 90 minutes. Every scan fails at roughly the 60-minute mark with expired credentials. The team raises `MaxSessionDuration` on the audit roles to 12 hours and requests `DurationSeconds` of 43200; the `AssumeRole` call now fails outright.

What is happening, and what should the team do?

- A `MaxSessionDuration` cannot exceed 6 hours for cross-account roles; request 21600 instead
- B This is role chaining, capped at 1 hour; assume the audit roles directly from a non-chained identity
- C The instance profile credentials expire hourly; shorten the scan into 45-minute batches
- D Add `sts:TagSession` to the trust policies so the session can be refreshed in place

◆ PAUSE AND DECIDE

What is happening, and what should the team do?

A Cross-account max is 6 hours

Invented. MaxSessionDuration goes to 12 hours regardless of account boundary. The 1-hour cap here comes from chaining, not from being cross-account.

C Batch the scan into 45-minute chunks

Works around the symptom and leaves a fragile design. It also multiplies the number of assume-role calls across 90 accounts.

B Role chaining, capped at 1 hour; assume directly instead

Correct. The instance role is already an assumed session, so assuming from it is chaining - hard-capped at 1 hour, and requesting more fails rather than truncating.

D Add sts:TagSession to refresh in place

TagSession attaches tags to a session. It has nothing to do with duration, and no STS call refreshes an existing session in place.

THE SIGNAL

'Fails at roughly 60 minutes' and 'the longer request now fails outright' are the two signatures of chaining together.

Developers in a build account hold a policy with `ec2:RunInstances` on all resources and `iam:PassRole` with Resource *. The intention was to let them launch instances with the standard application role.

A penetration test shows a developer launching an EC2 instance with the account's CloudFormation deployment role attached, then reading that role's credentials from the instance metadata service and creating an administrator user.

What is the MOST effective fix?

- ☐ A Require IMDSv2 with a hop limit of 1 on all instances in the account
- ☐ B Add a Deny on `iam:CreateUser` to the developers' policy
- ☐ C Scope `iam:PassRole` to the specific application role ARNs, with an `iam:PassedToService` condition
- ☐ D Move the CloudFormation deployment role into a separate account

◆ PAUSE AND DECIDE

What is the MOST effective fix?

A IMDSv2 with hop limit 1

Good hardening from module 16 and it does not help here. The developer legitimately controls the instance, so IMDSv2's session token is theirs to obtain.

C Scope PassRole to specific ARNs with iam:PassedToService

Correct. The escalation exists because PassRole is unscoped. Naming the permitted roles removes it, and PassedToService stops a Lambda role being attached to an instance.

B Deny iam:CreateUser

Blocks one thing the escalated role did. The developer still holds a role that can do anything else CloudFormation's deployment role can do.

D Move the deployment role to another account

Relocates this particular role and leaves the mechanism. Any other privileged role in the account remains passable.

THE SIGNAL

iam:PassRole with Resource * means 'you may become any role in this account'. Read it that way and the escalation is obvious.

Five things to carry forward

- ◆ **A role is two policies.** Denied when assuming is the trust policy; denied after is the identity policy.
- ◆ **Principal ending in :root delegates to the whole account,** not to its root user.
- ◆ **External ID for a third-party company. SourceArn for an AWS service.** Both answer 'acting for whom'.
- ◆ **Role chaining is capped at 1 hour** and asking for longer fails. Assume directly instead.
- ◆ **Unscoped iam:PassRole means 'become any role here'.** Scope it by ARN and by PassedToService.

ON THE EXAM

Next: human identity – IAM Identity Center, federation, MFA and Directory Service.